SOURCE-GROUNDED RESEARCH REVIEW

Shorter Lean proofs for Diophantine consistency

Shared proof kernels, reusable arithmetic, and compact checked certificates

Prepared for Vladimir Reshetnikov

26 October 2026

Reviewed repository

https://github.com/VladimirReshetnikov/ProveIt

`Computability/DiophantineConsistency/Lean`

Commit `60e9c5d620c6c387d932876cfa2d9376f64ea56c`

Lean 4.32.0; pinned mathlib commit

`81a5d257c8e410db227a6665ed08f64fea08e997`

Central finding. The strongest opportunities come from making existing abstractions carry more of the proof: assignment transport, scanner simulation, topological induction, and the circuit power checker. Small arithmetic improvements can be adopted independently. The largest source-volume opportunity lies in generated gate and power proofs, but it needs controlled certificate interfaces that preserve sharing and named intermediate expressions.

Concrete deliverable. The archive contains a checked refactoring of the representation constructors, independently compiled generic proof modules, compiler/axiom logs, reproducible source metrics, and a staged plan for the larger changes. Compiled examples and proposed integrations are identified separately.

This is a refactoring study of existing formal proofs. Its measurements refer to the pinned snapshot. It does not claim a completed rewrite of the entire project, a measured global speedup, or completion of the still-open formalization links documented by the repository.

Contents

1 What the review establishes **2**
1.1 Snapshot and coverage . . . 2
1.2 Recommended order . . . 3

2 Immediate refactorings with precise mathematical content **3**
2.1 Assignment transport and witness splitting . . . 3
2.2 Replace half-radix case enumeration by one identity . . . 4
2.3 Use the existing sum-at-most-one API . . . 5
2.4 Derive carry recurrence from normalized local arithmetic . . . 5
2.5 Use a stronger congruence instead of proving a weaker one again . . . 6
2.6 Lift pointwise zero-set equivalence once . . . 6

3 One simulation kernel for the annotated scanners **7**
3.1 The repeated structure . . . 7
3.2 The checked abstraction . . . 7
3.3 How to migrate without changing coded programs . . . 7

4 The largest opportunity: compact circuit proof certificates **8**
4.1 Measure proof schemas separately from literal circuit data . . . 8
4.2 Reuse the existing power checker systematically . . . 8
4.3 One topological induction principle . . . 9
4.4 Replace thousands of gate-proof lines without unfolding the evaluator . . . 9
4.5 Factor the two translation proofs and admit commutative swaps . . . 9
4.6 Share residual packaging before merging quartic compilers . . . 10
4.7 Use low-degree row certificates as the first reflection experiment . . . 10
4.8 Preserve cancellation-aware degree proofs . . . 11

5 Word arithmetic and geometric interfaces **11**
5.1 Extract cyclic-field arithmetic below the geometry modules . . . 11
5.2 One origin-correction lemma for multiple mask generations . . . 12
5.3 Separate arbitrary folding from injective sparse placement . . . 12
5.4 A narrow tactic for coefficient envelopes . . . 12

6 Checker composition and internal arithmetic **13**
6.1 Reuse parser and list infrastructure that already exists . . . 13
6.2 Normalize finite-index reasoning in the object layer . . . 13
6.3 Wrap internal induction while retaining definability . . . 13
6.4 Keep the unresolved logical interfaces explicit . . . 14

7 A practical migration plan **14**
7.1 First change: foundational proof reuse . . . 14
7.2 Second change: shared induction and interfaces . . . 14
7.3 Third change: generated certificates . . . 15
7.4 What to measure . . . 15

8 Validation and contents of the archive **15**
8.1 What was compiled . . . 15
8.2 What remains an integration proposal . . . 16
8.3 Source map for the principal proposals . . . 17

1 What the review establishes

The most effective way to shorten this development is to reduce the number of independently maintained proof patterns. The code already has a strong separation between explicit arithmetic data, executable evaluators, finite certificates, and mathematical correctness. That separation should be retained. Several repeated arguments can be moved into the existing foundational layers without changing any polynomial, witness coordinate, primitive-recursive program, or statement of consistency.

There are three different outcomes in this report:

1. **A concrete source patch.** The representation constructors are shortened while preserving their complete `Rep` structures. The archive includes the patch, compatibility proofs, and validation evidence.
2. **Compiled proof prototypes.** General scanner, circuit, arithmetic, finite-support, and cyclic-residue results are checked with the pinned compiler. Some are used in replacement proofs against the original project modules. A compiled general theorem does not by itself establish that every proposed caller migration has been completed.
3. **Larger integration proposals.** Generated proof compression, complete translation certificates, object-level wrappers, and shared quartic packaging are grounded in specific source declarations. Their full implementation and net savings remain to be measured.

1.1 Snapshot and coverage

The review uses the main-branch snapshot at commit `60e9c5d620c6`, whose last change concerns cell translation, field offsets, and cycle exclusions. Source links in this report are pinned to the full commit. The active Lean subtree contains 546 Lean files. There are another 165 Lean files elsewhere under the requested directory, mostly preserved candidate or historical material; they are excluded from the active-source totals. The detailed audit concentrates on the named proof families below rather than claiming a line-by-line mathematical review of all 73,971 lines.

Measure in the active Lean subtree	Count
Lean source files	546
Physical lines	73,971
Nonblank source lines	63,808
Nonblank lines after lexical comment/string removal	59,832
Theorem/lemma declarations, lexical count	4,970
UTF-8 source bytes	3,293,801
Audit/check files, by documented naming/path rule	220
Physical lines in those audit/check files	7,114
Files with a generated-source header in the first 30 lines	18
Physical lines in those generated-header files	14,807

Table 1: Reproducible source measurements; these are not proof-term or elaboration-cost measurements.

The script `scripts/audit_lean.py` defines every counting rule and records per-file hashes. It supports nested Lean comments and blanks ordinary string literals before lexical counting. It found no byte-identical Lean files, but identified 29 candidate clusters of identical normalized tactic bodies. Those candidates require semantic inspection: identical text in different namespaces need not express the same theorem.

The source scan found no code tokens `sorry`, `admit`, `axiom`, or `native_decide` in the active subtree. This is useful inventory evidence, not an axiom audit of all imported results. The supplied compiled experiments additionally use `##print axioms`; their successful outputs are retained. The distinction follows Lean's documented separation between source elaboration, trusted logical axioms, and compiler-supported reduction.[1]

1.2 Recommended order

Order	Change	Reason to do it at this stage
1	Assignment and witness-block transport	A concrete checked patch; preserves literal data and reduces foundational repetition.
2	Half-radix identity, one-hot support, carry views	Small mathematical lemmas with checked prototypes and clear reuse.
3	Existing stronger residue theorem	A one-line application replaces a repeated finite case proof.
4	Shared scanner simulation	Four closely related modules can use one induction, while keeping raw-input soundness separate.
5	Indexed power certificates	Existing `run_power` and existing Split label data already provide most of the architecture.
6	Shared circuit invariant and translation relation	Checked generic results factor topology bookkeeping and harmless commutative swaps.
7	Local gate proof generation with explicit stopping points	Targets 999 repetitive gate equations and long rewrite lists without expanding entire circuits.
8	Low-level cyclic-field and mask helpers	Removes repeated arithmetic/support arguments and avoids import cycles.
9	Shared residual-system and sparse-row certificates	Reduces quartic and quadratic-row repetition, preserving independent specifications.
10	Object-level induction/validity wrappers	Simplifies proof presentation while retaining definability and nonstandard-model quantifiers.

2 Immediate refactorings with precise mathematical content

2.1 Assignment transport and witness splitting

Targets. In `Representation.lean`: `Rep.reindex` (lines 155–166), `Rep.conjunction` (170–198), and `Rep.disjunction` (202–231). The same technique can later support `Rep.existsBlock`. A narrower assignment-composition lemma already exists privately in `QuarticCompiler.lean`, lines 95–98. The proposed change promotes the idea into the foundational layer and makes both input and witness renaming explicit.

The common equation is

$$\mathrm{natAssignment}(v, w) \circ \mathrm{Sum.map}(f, g) = \mathrm{natAssignment}(v \circ f, w \circ g).$$

[1] https://lean-lang.org/doc/reference/latest/ValidatingProofs/ and https://lean-lang.org/theorem_proving_in_lean4/Axioms-and-Computation/. The actual experiments use Lean 4.32.0, rather than relying on whichever version the current documentation describes.

Its proof is just function extensionality followed by the two sum-type cases. Combined with the existing `Expr.eval_rename`, it removes the local assignment equalities repeated in the binary constructors.

The remaining bookkeeping concerns an assignment on `Fin (m+n)`. Prove once that

$$\exists w,\ P(w \circ \mathrm{castAdd}_n) \wedge Q(w \circ \mathrm{natAdd}_m) \\ \longleftrightarrow (\exists a, P(a)) \wedge (\exists b, Q(b)).$$

There is an analogous disjunction equivalence. Its reverse direction supplies zero to the unused natural witness block. This default witness matters: an analogous statement over arbitrary empty witness-value types would need an inhabitation assumption.

The resulting correctness proof for conjunction becomes:

```
intro v
simp only [left, right, Expr.eval_add, Expr.eval_mul, Expr.eval_rename,
  assignment_comp, Function.comp_id, mul_self_add_mul_self_eq_zero]
exact (and_congr (r.correct v) (s.correct v)).trans
  (exists_fin_add_and _ _).symm
```

The constructor still supplies exactly the same witness count and exactly the same expression tree. In the checked prototype, the strongest compatibility assertions are:

```
theorem reindexShort_eq ... : reindexShort r f = r.reindex f := rfl
theorem conjunctionShort_eq ... : conjunctionShort r s = r.conjunction s := rfl
theorem disjunctionShort_eq ... : disjunctionShort r s = r.disjunction s := rfl
```

Here the ellipses abbreviate binders in the exposition only; complete compilable declarations are in `lean/AssignmentTransport.lean`. Definitional equality of the data, together with proof irrelevance for correctness fields, proves equality of the whole structures. This is stronger than proving only that the new expression has the same existential zero set.

Honest accounting. The original three constructor declarations occupy 71 physical lines; their shortened versions occupy 22. The shared helpers cost 28 declaration lines. The complete source patch, including comments and blank lines, changes `Representation.lean` from 260 to 244 lines: 42 insertions and 58 deletions, a net reduction of 16 lines. The dramatic reduction at each call site should not be presented as a 69% reduction of the file or project. The helper's main advantage is that future witness constructions inherit a stable interface.

Extension. Add the nested-sum transport identity used by `existsBlock`; use the same helper in the semiring gate interpreter after generalizing its target value type. Keep the helper low in the import graph. Importing `QuarticCompiler` back into `Representation` would create the wrong dependency direction.

2.2 Replace half-radix case enumeration by one identity

Targets. In `NativeCyclicRotation.lean`: the bound theorem `shiftDigits_half_bound` (100–116) and the parity theorem `shiftDigits_half_parity_89` (160–165).

For $s > 0$ and arbitrary natural digits a, b,

$$(sa) \bmod (2s) + \left\lfloor \frac{sb}{2s} \right\rfloor = s(a \bmod 2) + \left\lfloor \frac{b}{2} \right\rfloor. \tag{1}$$

This is the exact mathematical content of the half-radix branch. Lean's core multiplication/remainder and multiplication/division lemmas prove it directly:

```
theorem half_digit_eq {s : ℕ} (hs : 0 < s) (a b : ℕ) :
    s * a % (2 * s) + s * b / (2 * s) = s * (a % 2) + b / 2 := by
  rw [Nat.mul_comm 2 s, Nat.mul_mod_mul_left,
    Nat.mul_div_mul_left _ _ hs]
```

If $b \le 3$, then $a \bmod 2 \le 1$ and $b/2 \le 1$, so the expression is at most $s + 1$. If s is even, its parity is $b/2$. No bound on the current digit a is needed. The old APIs may keep the stronger hypothesis that all digits are at most three; compatibility need not wait for a separate hypothesis cleanup.

The bound's original proof body has 15 nonblank lines and a four-way digit split; its adapter has one line. The parity proof has four body lines but generates sixteen cases; its adapter has two lines. The three shared scalar lemmas cost ten nonblank proof-body lines in the checked version. This gives an explicit account of the helper overhead and a genuine simplification of the argument, without asserting an unmeasured runtime gain.

The scalar lemmas do not inspect the indexing type. Consequently the cyclic predecessor $k - 1$ is retained exactly, including wraparound and any degenerate modulus allowed by the original theorem. There is no assumption of a zero incoming carry or an already aligned temporal phase. Artifacts: `HalfRadixCore.lean` and the project-definition adapters in `HalfRadixRefactor.lean`.

2.3 Use the existing sum-at-most-one API

Target. `CurrentCellLabels2011.lean`, `sum_le_one_cases` (131–158). The existing proof chooses a positive term, compares it with the sum, erases its index, proves the remaining sum is zero, and bounds each remaining term.

The core fact is already packaged by mathlib as `Finset.sum_le_one_iff`; the repository itself uses it in `NativeSelectorClause54.lean`, `selectors_one` (65–80). For a finite family $f_i \in \mathbb{N}$, it yields uniqueness of any nonzero index and that its value is one. Split only on whether all entries are zero. If not, choose a nonzero index and apply the uniqueness conclusion.

The supplied `OneHotCore.lean` proves the same generic statement using eight nonblank proof-body lines instead of 26. It reproduces the existing `Selected` predicate in a fresh namespace for isolated checking; production should simply replace the original proof body. Empty index types are covered by the all-zero alternative. A Boolean-input premise is unnecessary because the sum bound already supplies the required information.

Keep `Selected.unique` and `Selected.weighted_sum`. These are useful downstream interfaces; replacing them with repeated raw sums would undo the improvement.

2.4 Derive carry recurrence from normalized local arithmetic

Target. `CurrentPolynomialWindow2011.lean`, the `Facts` fields near lines 58–66.

Suppose natural numbers satisfy

$$\text{raw} = \text{digit} + \rho\,\text{carry}, \qquad \text{digit} < \rho.$$

Then

$$\text{carry} = \text{raw}/\rho, \qquad \text{digit} = \text{raw} \bmod \rho.$$

The digit bound implies $\rho > 0$, so both conclusions follow directly from division and remainder laws. `CarryCore.lean` checks the two short proofs.

The current facts record stores the digit bound, local carry equation, and quotient recurrence. Store the first two and expose the recurrence as a theorem named `Facts.carry_recurrence`. Existing client notation `h.carry_recurrence` can remain natural, while constructors lose a redundant obligation. A small normalized-digit structure can support both views pointwise.

This refactoring applies only after the digit bound is available. Some early geometry proofs intentionally assume just a local equality. They cannot recover quotient information from that weaker interface. Also retain the existing generic fixed-point construction in `NativeCyclicCarry.lean`: a linear induction starting from zero carry is not a replacement for a cyclic-word argument.

2.5 Use a stronger congruence instead of proving a weaker one again

Target. `CurrentEarlyPadding2011.lean`, `reflected_mod_sixteen` (99–107). It repeats the same selector/marker case split already performed in `CurrentClause2011.lean`, `reflected_weight_mod` (122–131), with a smaller modulus.

Since 16 | 2048, the complete replacement is

```
exact (CurrentClause2011.reflected_weight_mod e).of_dvd
  (by decide : 16 | 2048)
```

The two private selector indicators have the same definition, so the target expressions are definitionally equal. A later cleanup can move that indicator next to `Entry`, but it is not a prerequisite for this proof reuse. The original proof has eight nonblank body lines; the replacement is one logical application. More significantly, one finite case argument disappears while all actual coefficient cases remain certified by the stronger theorem.

2.6 Lift pointwise zero-set equivalence once

Target. `CurrentCarrierArithmetic2011.lean`, `positive_zero_69_iff_70` and `positive_zero_70_iff_71` (95–119).

The pointwise polynomial result already compares the same positive assignment. A congruence for the existing `PositiveZero` predicate lifts it through the existential quantifier and positivity guard:

```
theorem positiveZero_congr {n : ℕ} (P Q : MvPolynomial (Fin n) ℤ)
    (h : ∀ u : Fin n → ℤ, (∀ j, 0 < u j) →
      (MvPolynomial.eval u P = 0 ↔ MvPolynomial.eval u Q = 0)) :
    PositiveZero P ↔ PositiveZero Q := by
  exact exists_congr fun u => and_congr_right (h u)
```

The two existing equivalences then become applications of the pointwise result. The first port's positivity can also have one named lemma instead of three repeated cast proofs.

This helper does not replace the $65 \leftrightarrow 69$ theorem: that result changes the witness dimension from fourteen to thirteen and needs its actual `completes`/`restores` maps. Preserve the fact that the 69/70/71 comparisons impose no positivity condition on K. Adding such a hypothesis merely to unify theorem signatures would weaken the API.

3 One simulation kernel for the annotated scanners

3.1 The repeated structure

The main loop argument appears in `AnnotatedRenameScan.lean` (29–87), `AnnotatedBoundScan.lean` (25–83), `AnnotatedLineScan.lean` (32–54 and 131–154), and `AnnotatedAxiomContext.lean` (30–50 and 130–151). `AnnotatedContext.lean` shares the iteration/fixed-point part but has a different terminal relation.

Each numeric scanner operates on a state encoding a remaining list and a history. Rejection resets the state to zero. An empty remaining list is stable. A successful row advances to the tail and updates history. The semantic list runner has the shape

$$\begin{aligned} \mathrm{run}([], h) &= \mathrm{some}(h), \\ \mathrm{run}(a :: as, h) &= \begin{cases} \mathrm{run}(as, \mathrm{push}(a, h)), & \mathrm{valid}(a, h), \\ \mathrm{none}, & \text{otherwise.} \end{cases} \end{aligned}$$

The payload update must be a parameter. Renaming and bound scans retain a raw row; line and axiom-context scans can retain a checked output. Hard-coding cons of the input row would miss those instances.

3.2 The checked abstraction

`GenericScanner.lean` needs only `import Std`. Its implementation record carries a numeric step, an encoding of remaining rows and history, and three laws:

1. The rejecting state is fixed: $\mathrm{step}(0) = 0$.
2. Encoded empty lists are fixed.
3. One encoded cons step is the corresponding semantic branch.

One theorem then proves that, for fuel at least the remaining list length, the numeric iteration equals zero on semantic failure and the encoded final history on success. Extra fuel is handled by the fixed-point law. The theorem accepts terminal/reject encoding collisions because it is a simulation statement; a separate finishing predicate decides whether the terminal history is acceptable.

The module also proves preservation of an arbitrary inductive history invariant and preservation of a raw numeric-state invariant under iteration. Its axiom audit records only `propext` for the list simulation and no axioms for the two invariant theorems.

3.3 How to migrate without changing coded programs

Keep the existing numeric `step`, `scan`, and supplied primitive-recursive programs. Construct an instance of the generic implementation using each module's existing local arithmetic theorem. Prove the required equality between the generic and existing list runner. Replace the repeated simulation proof with the shared result and retain the existing public theorem name.

This is intentionally a proof-level extraction. Replacing the checker definitions themselves may change their syntax trees or node counts, even when the functions are extensionally equal. The project has literal program-size certificates; proof simplification should preserve that data unless a separate program optimization is intended.

Raw-input soundness remains a separate obligation. In particular, `AnnotatedLineScan.lean`'s soundness theorem for arbitrary natural input codes does more than simulate the encoding of a known well-formed list. Its local checker reconstructs genuine provable judgments. The generic raw-state invariant can organize that proof, but the reconstruction argument cannot be omitted or replaced by a well-formed-input assumption.

4 The largest opportunity: compact circuit proof certificates

4.1 Measure proof schemas separately from literal circuit data

File	Physical lines	Gate lemmas	Power lemmas
`SplitCircuitSteps59.lean`	3,445	515	107
`GapCircuitSteps66.lean`	3,122	484	86
Total	6,567	999	193

Table 3: The two dominant generated proof-schema files.

These two files occupy 8.88% of all physical lines in the active subtree. Their local-gate declaration blocks account for 4,995 lines and their power blocks for 1,540. These are exposed targets for refactoring, not achieved savings. A replacement must charge its certificate data, generic checker, compatibility aliases, and revised consumers.

By contrast, the large lists in `SplitCircuit59.lean`, `SplitJointCircuit59.lean`, and `GapCircuit66.lean` are actual circuit data. Moving those lists or hiding them in another file does not simplify a mathematical proof. The intended reduction is in independently generated reasoning and manually maintained dependency traversal.

4.2 Reuse the existing power checker systematically

`ArithmeticDAGPowers.lean` already defines `PowerStep` and proves `run_power` for arbitrary commutative rings. A nonzero label is an exponent. At a multiplication node, checked labels satisfy

$$\ell_i = \ell_a + \ell_b,$$

and the base register has label one. Topological induction proves that every labeled register is the appropriate power of the base.

Yet the two step files separately prove 193 power identities by rewriting predecessor identities and invoking polynomial normalization. More concretely, `SplitJointPower59.lean` already contains the parent Split circuit's power-label table and finite certificate. Move that parent certificate into a small independent module rather than importing the entire joint child circuit merely to obtain it. Generate and check the analogous Gap labels.

Expose one indexed theorem. Initially preserve old `natural_power_*` names as thin aliases so existing explicit rewrite lists and audits remain valid. Then migrate consumers to the indexed result or a controlled local tactic. The Python generators already calculate the exponent labels; the generated data should be checked in Lean with the existing kernel-based finite-checking approach. The generator is not trusted to establish the theorem.

The supplied `DAGRefactoring.lean` includes a checked adapter that accepts a finite exponent certificate and invokes `run_power`. The full replacement of the 193 named declarations remains an integration task. It should change `generate_split_recipe_steps59.py` and `generate_gap_recipe_steps66.py` as well as their emitted Lean files.

4.3 One topological induction principle

The same induction appears in `ArithmeticDAGSemantics.lean`, `run_eq_of_node_equations` (97–109); `ArithmeticDAGDegree.lean`, `value_degree_le` (65–82); `ArithmeticDAGPowers.lean`, `run_power`; and `NaturalDAGQuartic.lean`, `run_nonnegative` (43–53).

A generic invariant $P(i, v)$ should express what is known about register i and its value. The step theorem receives the invariant for every referenced predecessor and proves it for the current node's evaluated right-hand side. The shared proof handles strict predecessor bounds and rewrites the evaluator equation once.

Instantiations include equality to a proposed value table, a polynomial-degree bound, nonnegativity, an interval bound, or a power label. `DAGRefactoring.lean` checks this induction principle and same-signature shortened forms of evaluator uniqueness and degree soundness. This is a modest immediate line improvement with a substantial interface benefit: future invariants no longer need to know the list evaluator's indexing proof.

Do not expect a net saving from adding a generic theorem and migrating only one tiny consumer. Migrate the closely related family together, while retaining local arithmetic lemmas as the semantic content of each instance.

4.4 Replace thousands of gate-proof lines without unfolding the evaluator

Each of the 999 local gate equations is a specialization of the existing `run_node_equation`. A small proof-producing tactic can construct such an equation on demand: identify the literal register index, invoke the generic theorem, prove the finite index bound, and reduce just the selected node and `Node.evalAt`.

The stopping rule is essential. Leave predecessor registers as named expressions until a caller asks to expand them. Global simplification of `run` or recursive expansion of every register into a polynomial loses the very sharing that makes a DAG compact.

The present register proofs show why this matters. A line in `GapCircuitRegisters66.lean` contains 2,142 characters and 125 explicit step/power references. The two register modules together contain 660 step references and 254 power references. These long dependency traversals are mechanical; the named base, kernel, threshold, and finalizer values are useful mathematical boundaries.

A suitable tactic or checker should accept semantic stopping registers, use the power certificate before traversing a power chain, cache intermediate equations, and return ordinary proof terms. Maintain a small explicit specification of the stopping points. A tactic that merely hides the original enormous expansion in one command would reduce visible text but might worsen elaboration, term size, and debuggability.

4.5 Factor the two translation proofs and admit commutative swaps

`SplitCircuitTranslation59.lean` and `GapCircuitTranslation66.lean` each have 79 lines. Their full text matches after documented family and finite-parameter substitutions; `metrics/focused-circuits.json`

reproduces the comparison. Both use the same retained-register map, the same private input-irrelevance lemma, and one special case for a multiplication whose operands are reversed.

Extend the existing node relation with a commutative alternative:

$$\mathrm{Rel}_{+/*}((a,b),(c,d)) \iff (a \sim c \wedge b \sim d) \\ \vee (a \sim d \wedge b \sim c).$$

Subtraction must retain ordered operands. The compiled `NodeRelComm` and `evalAt_of_NodeRelComm` establish this exact distinction. They remove the reason for the isolated swapped-product proof cases.

A complete shared translation theorem can then take two node lists, their topology proofs, the retained-register map, an explicit initial-input interface, and a finite correspondence certificate. Each family supplies its actual circuit data. The special handling of the unshifted stored input is a real semantic feature and remains explicit; it is not removed by admitting a commutative operand swap.

The generic relation and soundness proof are checked. The complete two-family migration is proposed work, not a claimed completed simplification of both translation modules.

4.6 Share residual packaging before merging quartic compilers

`DAGQuartic.lean` and `NaturalDAGQuartic.lean` duplicate sum-expression evaluation, sum-of-squares reasoning, degree helpers, and an identical 19-line `rhs_degree` body. The former also has a general private `sumExpressions_cost` theorem with an arbitrary per-expression bound, whereas `QuarticCost.lean` repeats essentially the same induction for bound five.

The first extraction should be a finite residual-system API. For residual expressions r_i over integer values, prove once:

$$\sum_i r_i(v)^2 = 0 \iff \forall i,\ r_i(v) = 0.$$

If every residual polynomial has degree at most two, prove the summed-square polynomial has degree at most four. Give a syntax-operation bound parameterized by the cost of each residual. Promote the already existing general cost theorem and retain the old bound-five result as a specialization.

The shared six-constructor right-hand-side proof can be abstracted over the expressions representing inputs and wires. A later wire-codec abstraction may capture both full compilers, but their realization conditions must remain distinct. The signed compiler encodes arbitrary integers using two natural coordinates; the one-coordinate compiler requires a genuine nonnegativity invariant. Preserve the exact counts inputs + 2 nodes and inputs + nodes and their separate operation bounds.

4.7 Use low-degree row certificates as the first reflection experiment

`GapCertificate66.lean`, `SplitCertificate59.lean`, and `NativeQuadraticParent.lean` each contain 22 individual row-identity proofs. Their identity sections total 1,074 physical lines, followed by essentially the same 22-case dispatcher. These rows are quadratic, so they offer a more bounded normalization problem than a complete high-degree circuit.

Use a sparse row syntax for constant, linear, and quadratic terms, with the fixed ports as coefficient parameters. Prove evaluation of that syntax once. A checked normalizer or a proof-producing tactic can then certify each generated row against its independently specified source row. The final dispatcher becomes table-driven.

The independent source specification must remain. Redefining `deliveredRows` or `shiftedRows` to be the generated encoding would erase the connection that the current identities prove. Likewise, random numerical evaluation cannot establish a polynomial identity. Retain universal quantification in the fixed ports and coordinate values, and initially preserve public row-theorem names as aliases.

4.8 Preserve cancellation-aware degree proofs

The project already distinguishes structural upper bounds from sharper algebraic bounds. `GapCircuitDegree66.lean` obtains the generic bound 2,063,116. `GapFinalizerDegree66.lean`, together with `FreeCarrierDegree.lean`, obtains the stronger bound 1,845,940 by using actual identities and Pell-related cancellations.

A shorter proof of the larger bound is not a replacement for the smaller one. The useful extension is a checked rewrite block: first prove that a compact rewritten expression equals the actual circuit expression, then apply the existing weighted-degree machinery to that normal form. Keep all required port conditions and named-register equalities visible. Numeric degree labels certify upper bounds; they do not automatically prove exact degree or the absence of leading-term cancellation.

5 Word arithmetic and geometric interfaces

5.1 Extract cyclic-field arithmetic below the geometry modules

`CurrentClause2011.lean` (65–94) and `CurrentPhaseParity2011.lean` (49–78) repeat about thirty lines of the same calculation: determine two source residues, prove that they lie outside the low support, and conclude that the corresponding source digits vanish. Their target fields differ, but their arithmetic pattern does not.

The compiled `CyclicFieldRefactor.lean` works in the full ambient group $\mathbb{Z}/(NL)\mathbb{Z}$, with $N, L > 0$ (the code has `NeZero (N * L)` and an explicit proof of $L > 0$). For an aligned base b, it proves

$$\mathrm{val}(b + e - t) \bmod L = (L + e - (t \bmod L)) \bmod L. \tag{2}$$

The offset e and shift t are arbitrary natural numbers; the concrete upper bound on e used in a current theorem is unnecessary. A companion support lemma turns a proof that the right side is outside a support set into a zero source digit.

The existing `CurrentCellTranslation2011.lean`, `source_residue` (92–105), already has the concrete version. Importing it into `CurrentClause2011` would create a cycle, because the translation module depends on the clause module. Extract the common addition, subtraction, and source-residue lemmas into a genuinely low-level module, and retain concrete wrappers under old names.

Only the field residue forgets whole-cell motion in equation (2). The group element itself still uses the entire shift t. Replacing that group element by a shift of $t \bmod L$ loses temporal translation and is incorrect. The generic theorem preserves the $N = 1$ case as well.

The actual support is an `irreducible_def`. Closed exclusions should use its existing characterization lemma, for example `norm_num [CurrentMasks2011.lowSupport_def]`, rather than assuming that a bare decision tactic will unfold the hidden definition.

5.2 One origin-correction lemma for multiple mask generations

NativeHighPort54.lean, mask_word (183–203), and CurrentHighDigits2011.lean, high_mask_word (107–126), both subtract one from the origin digit of a packed word and repeat the cancellation proof.

For any radix r, finite cyclic word a, and $q \le a_0$,

$$\mathrm{word}_r(k \mapsto a_k - q[\,k = 0\,]) = \mathrm{word}_r(a) - q.$$

Prove the pointwise addition identity, apply word additivity and the existing origin-word theorem, and cancel. The guard $q \le a_0$ is necessary because subtraction in $\mathbb{N}$ is truncated. There is no digit-upper-bound or radix-positivity premise in this argument. The supplied WordOriginRefactor.lean makes this proposed reusable API concrete; its exact validation classification appears in the experiment inventory.

After extraction, each mask proof needs to establish only its own periodic repetition and single-cell formula. Existing additive origin-mask lemmas remain useful and can share the same foundational layer.

5.3 Separate arbitrary folding from injective sparse placement

NativeCyclicConvolution.lean already has foldStencil, mass_foldStencil, word_foldStencil, and convolution_foldStencil. CurrentKernel2011.lean already uses them effectively. Rebuilding that API would add machinery without addressing the actual repetition.

The remaining opportunity is the injective-placement specialization used in NativeHighPort54.lean and NativeWordPopulation.lean. A sparse table with an embedding into digit positions can expose lookup at an image, lookup outside the image, coefficient bounds, word evaluation, and population additivity. Natural positions already known to lie below n can be stored as Fin n and sent through ZMod.finEquiv n, avoiding repeated cast-injectivity proofs.

Do not assume injectivity for arbitrary folded stencils: exponents may collide and their coefficients must add. Also distinguish exact word evaluation at bounded positions from evaluation at arbitrary exponents modulo $r^n - 1$. The latter generally gives a congruence, not equality of natural representatives.

5.4 A narrow tactic for coefficient envelopes

NativeCoefficientBounds.lean, lines 63–600, contains 22 rowEnvelope_* proofs. They repeatedly introduce the same nonnegative monomials, split nested conditional coefficients with abs_ite_le, and solve the leaves with the same linear-arithmetic context.

A small local tactic can recurse only through those conditional bounds and use an explicitly named set of nonnegativity facts at the leaves. A later signed-coefficient certificate could share the sparse-row infrastructure rather than creating a second unrelated normalizer.

Preserve the actual envelope and its dependence on the large parameter K. A simpler but much looser estimate, especially one quadratic in K where the existing proof is linear, can change the construction's size argument. This proposal targets proof repetition, not the bound being proved.

6 Checker composition and internal arithmetic

6.1 Reuse parser and list infrastructure that already exists

`AnnotatedProofCode.lean` repeats a parse-option split followed by typed checker soundness in `check_sound`, goal-specific checking, and refutation checking (around 402–459). One small parse-then-check lemma should extract a successfully parsed certificate and its successful typed check. Specialized soundness follows by composition with `Certificate.check_sound`.

Similar quote-injectivity proofs occur for trees, judgments, contexts, and formulas. They follow from a partial decoder's left-inverse equation $\mathrm{decode}(\mathrm{encode}(a)) = \mathrm{some}(a)$. A common helper can express that once. This proves injectivity of valid encodings, not that every natural number parses as a formula. The total decoder for a six-field row is a separate representation theorem.

An even smaller change needs no new helper: `AnnotatedProofChecker.lean`, `mem_of_lookup` (177–187), reproves a list fact available in the pinned Lean core as `List.mem_of_getElem?`. Replace the local induction and its use in line soundness with that existing theorem. This is a good preliminary patch because it removes repetition without adding an abstraction.

The seventeen rule-coordinate cases in `AnnotatedRuleSchedules.lean` are already compact `fin_cases`/`rfl` proofs. A metaprogram for them is unlikely to improve clarity. Likewise, `AnnotatedSchemaMaps.lean` already has a useful chain interface. Its nonempty-chain premise in soundness must remain: the empty chain merely compares raw codes and does not validate formula syntax.

6.2 Normalize finite-index reasoning in the object layer

`ObjectQuadraticSystem.lean`, `conjunction_rows_zero_iff` (17–27), manually decomposes a universal quantifier over `Fin (m+n)`. The project already uses `Fin.forall_fin_add` elsewhere. Use that equivalence together with the `Fin.addCases` reduction lemmas.

`ObjectDivModSyntax.lean`, `rows_zero_iff` (21–39), and `ObjectPredecessorSyntax.lean`, the corresponding theorem (24–41), establish two row facts and then manually reconstruct a two-element universal statement. The existing `Fin.forall_fin_two` can replace the final conjunction plumbing. These are source-grounded replacement suggestions; the full object-layer integrations are not claimed as compiled experiments in this archive.

The statements must still range over `ObjectOmegaSemiring.Natural H`. Substituting ordinary Lean naturals would remove the internal-model content of the theorem. Assignment composition can likewise be factored at the semantic value type rather than specialized back to standard naturals.

6.3 Wrap internal induction while retaining definability

`ObjectAdditionLaws.lean` and `ObjectMultiplicationLaws.lean` repeatedly construct an object formula, prove its satisfaction specification, apply `omega_ind`, and translate base cases, induction hypotheses, and conclusions with the two directions of that specification.

A useful wrapper takes an actual object formula φ, an environment e, a semantic predicate P, and a proof

$$\forall n, \quad \mathrm{Sat}(\mathrm{scons}(n, e), \varphi) \longleftrightarrow P(n).$$

It then takes the zero and successor arguments and returns $P(n)$ for all n in the model's ω. The formula and satisfaction equivalence are essential. A wrapper accepting an arbitrary external P with

no definability evidence would assert an unjustified induction principle for a nonstandard model.

`ObjectCodeBound.lean` already demonstrates the right discipline: its form-code induction takes explicit object predicates and their satisfaction equivalences. Extend that approach to the repeated natural-number inductions.

The semantic-validity-to-provability boundary can also have a wrapper around the existing completeness theorem, the sentence condition, and the model's ZFC axioms. It must still prove validity of an explicit object formula in every relevant model. A standard-natural-number argument is not a replacement.

6.4 Keep the unresolved logical interfaces explicit

The project documents separate layers: external correctness on standard natural codes, internal semantics over model naturals, quoted formal derivations, and the complete native polynomial-to-machine connection. `ObjectPolynomial.lean` (89–96) explicitly supplies one direction from internal unsolvability to external unsolvability. `ObjectConsistency.lean` records the outstanding internal sealed/unsealed proof-code translation.

The current `README.md` and `LEAN_EQUIVALENCE_PLAN.md` also distinguish completed arithmetic checkpoints from the remaining native and internal-ZFC composition. Refactoring should make those interfaces easier to prove and review; it must not replace a missing interface with a premise that already asserts the desired conclusion. None of the compiled helpers in this report completes those open links.

7 A practical migration plan

7.1 First change: foundational proof reuse

Run the included validation script in an untouched checkout at the pinned commit, with its pinned toolchain and dependencies. It extracts and compiles the original representation source before checking the old/new compatibility proofs, then compiles the patched source to a separate output path. Only after that check should the patch be applied in a disposable branch. The supplied patch passes a dry-run application check, and the prototype proves equality of the complete old and new constructor results. The successful logs in the archive state which additional examples were checked during this review.

Then adopt the low-risk arithmetic changes independently: half-radix identity, finite-support theorem reuse, normalized carry views, and the stronger congruence. Keep existing theorem names as wrappers. Give each generic helper a narrow import list where practical; shrinking imports should follow actual dependency checks rather than guesses about which tactics happen to be re-exported.

7.2 Second change: shared induction and interfaces

Migrate the four scanner simulations as a family. Prove each local adapter against its unchanged numeric implementation, and separately retain raw-input soundness and finishing-predicate correctness. Add the generic circuit invariant and migrate equality, degree, and nonnegativity consumers together.

Extract cyclic-field arithmetic into a low-level module before using it in clause and marker proofs. This resolves the import-direction problem and keeps the full ambient modulus visible. Consolidate the origin-word correction only after checking its natural-subtraction guard at every caller.

7.3 Third change: generated certificates

Start with power labels because the generic proof and part of the data already exist. Update the generator and generated output together, expose an indexed theorem, and preserve legacy names until consumers have moved. Next migrate one translation family to a generic correspondence theorem with the commutative node relation. Retain actual input conversion and finite topology checks.

For local gate proof generation, begin with one named register block. Check that the resulting theorem has the same type and expected axiom dependencies. Measure elaboration cost and exported declaration size before removing the old generated file. Only then extend the approach across the whole finalizer recipe.

Low-degree row certificates are the next bounded experiment. A general compiler over typed or scoped DAGs is a longer-term option, but it should retain the existing flat literal data and prove conversion preserves its evaluator under `Topological`. The current total evaluator gives zero for invalid references; a scoped representation cannot silently replace its theorems for arbitrary lists.

7.4 What to measure

Report the following separately:

- Human-maintained source and generated source, including helper and certificate overhead.
- Number of exported declarations and sizes of generated proof modules.
- Elaboration time, peak memory, and compiled artifact size on a controlled machine.
- Exact theorem types, axiom dependencies, and preservation of literal construction data.

This review makes no performance claim based on wall-clock times from the shared execution environment. A smaller source file may still elaborate more slowly, and a one-line tactic may generate a very large proof term. Likewise, removing audit files or moving historical sources outside a counted directory is not a proof simplification.

8 Validation and contents of the archive

8.1 What was compiled

All twelve supplied experiment modules were compiled successfully with Lean 4.32.0 against the pinned project and mathlib environment. The complete patched `Representation.lean` was also compiled, and `git apply --check` accepted its patch. The compatibility module imports an isolated copy of the unchanged original; the patched module has a different output path.

Compiled artifact	Scope of the successful check
`AssignmentTransport.lean`	General transport and witness-splitting helpers; three complete old/new constructor equalities proved by `rfl`.
`GenericScanner.lean`	General simulation and invariant theorems. The four existing scanners have not been migrated.
`DAGRefactoring.lean`	Generic topological invariant, equality and degree consumers, input irrelevance, commutative node-relation soundness, and a power-certificate adapter.

Compiled artifact	Scope of the successful check
`HalfRadixCore.lean`	Scalar half-radix identity and bound/parity consequences.
`HalfRadixRefactor.lean`	Same-signature replacements for both original cyclic-rotation results, importing the original project module.
`OneHotCore.lean`	The same generic one-hot statement, with the original predicate reproduced in an isolated namespace.
`CarryCore.lean`	Quotient and remainder consequences of a normalized local carry equation.
`CyclicFieldRefactor.lean`	Addition/subtraction residues and support exclusion in the full cyclic ambient group.
`CyclicFieldProjectAdapter.lean`	All four concrete low-support exclusions from the original clause/marker arguments, grouped into two results.
`PositiveZeroRefactor.lean`	General congruence and both $69 \leftrightarrow 70$ and $70 \leftrightarrow 71$ existence comparisons against the actual project polynomials.
`ReflectedResidueRefactor.lean`	The modulus-16 replacement derived directly from the original modulus-2048 theorem.
`WordOriginRefactor.lean`	Subtraction-at-origin helper using the actual packed-word definitions and its required natural-subtraction guard.

The portable `scripts/verify.py --full` replay checks all twelve modules, the original representation module, patch applicability, and the complete patched module. It records commands, exit codes, source hashes, log hashes, and printed axiom dependencies in `validation/replay/status.json`. The higher-level classification is in `validation/status.json`. The earlier command transcript records the review machine's paths; the Python script is the portable entry point.

Every supplied successful printed axiom audit reports a subset of `propext`, `Classical.choice`, and `Quot.sound`; several invariant results use no axioms. No printed audit contains `sorryAx` or a native-evaluation axiom. This statement concerns the recorded elaborated declarations, not an exhaustive axiom audit of all 546 project files.

8.2 What remains an integration proposal

The archive does not migrate every scanner, replace both complete circuit translations, remove the generated power or gate declarations, implement the sparse-row certificate generator, or refactor the object-language induction layer. The checked helpers establish concrete reusable proof interfaces for those changes. Their total source reduction and elaboration costs must be measured after integration.

Selected original dependency targets were built successfully; the entire repository was not rebuilt. Existing unrelated linter warnings remain in the dependency logs. No global build-time or memory improvement is claimed. Source metrics are independently reproducible with `scripts/audit_lean.py`.

The archive includes the LaTeX/PDF report, checked Lean sources, the representation patch, source metrics and hashes, pinned environment files, and compiler/axiom logs. The README gives replay instructions.

The repository checkout was used for reading and isolated builds. The reviewed upstream branch was not changed. The patch is provided as a concrete reviewable artifact, so adoption can be a small, well-scoped repository change rather than an all-at-once rewrite.

8.3 Source map for the principal proposals

Topic	Pinned source anchors
Representation transport	`Representation.lean`, 155–256; `QuarticCompiler.lean`, 95–98.
Half-radix arithmetic	`NativeCyclicRotation.lean`, 100–116 and 160–165.
One-hot support	`CurrentCellLabels2011.lean`, 131–158; `NativeSelectorClause54.lean`, 65–80.
Carry normalization	`CurrentPolynomialWindow2011.lean`, 58–66.
Residue reuse	`CurrentClause2011.lean`, 118–131; `CurrentEarlyPadding2011.lean`, 95–107.
Positive zeros	`CurrentCarrierArithmetic2011.lean`, 78–119.
Scanner simulation	`AnnotatedRenameScan.lean`, 29–87; `AnnotatedBoundScan.lean`, 25–83; `AnnotatedLineScan.lean`, 32–54 and 131–154; `AnnotatedAxiomContext.lean`, 30–50 and 130–151.
Generic circuit semantics	`ArithmeticDAGSemantics.lean`, 58–116; `ArithmeticDAGDegree.lean`, 65–88; `ArithmeticDAGPowers.lean`, 7–61.
Generated powers	`SplitCircuitSteps59.lean`, 2591–3444; `GapCircuitSteps66.lean`, 2436–3121; `SplitJointPower59.lean`, 64–78.
Translation	`SplitCircuitTranslation59.lean`, 16–77; `GapCircuitTranslation66.lean`, 16–77.
Quartic packaging	`DAGQuartic.lean`, 83–96, 166–176, 199–252; `NaturalDAGQuartic.lean`, 110–122, 209–261; `QuarticCost.lean`, 65–74.
Sparse row identities	`GapCertificate66.lean`; `SplitCertificate59.lean`; `NativeQuadraticParent.lean`, the 22 row-identity blocks and their final dispatchers.
Sharp degree bounds	`GapCircuitDegree66.lean`, 50–66; `GapFinalizerDegree66.lean`, 42–70; `FreeCarrierDegree.lean`, 15–94.
Cyclic geometry	`CurrentClause2011.lean`, 65–94; `CurrentPhaseParity2011.lean`, 49–78; `CurrentCellTranslation2011.lean`, 92–105.
Origin masks	`NativeHighPort54.lean`, 183–203; `CurrentHighDigits2011.lean`, 107–126.
Internal arithmetic	`ObjectAdditionLaws.lean`; `ObjectMultiplicationLaws.lean`; `ObjectCodeBound.lean`, 324–328 and 367–370.
Finite internal rows	`ObjectQuadraticSystem.lean`, 17–27; `ObjectDivModSyntax.lean`, 21–39; `ObjectPredecessorSyntax.lean`, 24–41.
Logical status	`README.md`; `LEAN_EQUIVALENCE_PLAN.md`; `ObjectPolynomial.lean`, 89–96; `ObjectConsistency.lean`.